Informe del Caso de Estudio: Employee Attrition y Monthly Income

Integrantes: Jose Antonio Favian Yujra Yana
Leonardo Delgado Medrano
Alan Franz Flores Campos
Christian Coronel Condri
Juan Jose Cordeiro

Fecha: 24/2/2026

1. Planteamiento del problema y criterios de negocio

Contexto

El conjunto de datos contiene información de empleados de una empresa (WA_Fn-UseC_-HR-Employee-Attrition.csv). Se analizan dos problemas de negocio:

1. **Predicción del ingreso mensual (`MonthlyIncome`)**: un problema de regresión. Objetivo técnico: estimar cuánto debería ganar un empleado en función de su perfil.
2. **Predicción de Rotación (`Attrition`)**: un problema de clasificación binaria (Yes/No). Objetivo: detectar a los empleados con riesgo de renuncia.

Criterios de negocio y costos de error

- **Regresión**: un error absoluto grande en la predicción de sueldo puede implicar asignaciones presupuestarias ineficientes y descontento interno.
- **Clasificación**: se define la clase positiva como "el empleado renuncia". Los falsos negativos (FN) son especialmente costosos, ya que implican perder talento sin oportunidad de retención. Los falsos positivos (FP) generan costos de retención innecesarios.

Se asume que:

- Las características utilizadas están disponibles antes de la decisión de renuncia o el establecimiento del salario.
- El costo de un FN es mayor que el de un FP en el modelo de `Attrition`.

2. Exploración y calidad de datos

- No se detectaron valores nulos (`df.isnull().sum()` retorna ceros en todas las columnas).
- Se revisó la distribución de variables numéricas y categóricas mediante histogramas, boxplots y countplots estratificados por `Attrition`.
- Se examinó la matriz de correlación para entender relaciones y posibles fugas de información.
- Columnas constantes o irrelevantes:
  - `EmployeeCount`, `StandardHours`, `Over18` (valor único en todas las filas).
  - `EmployeeNumber` (identificador único, no predictivo). Estas columnas se descartaron antes del modelado.
- El desbalance en `Attrition` es marcado: ~16% "Yes", 84% "No".

Figura 1: Ejemplo de heatmap de correlaciones.

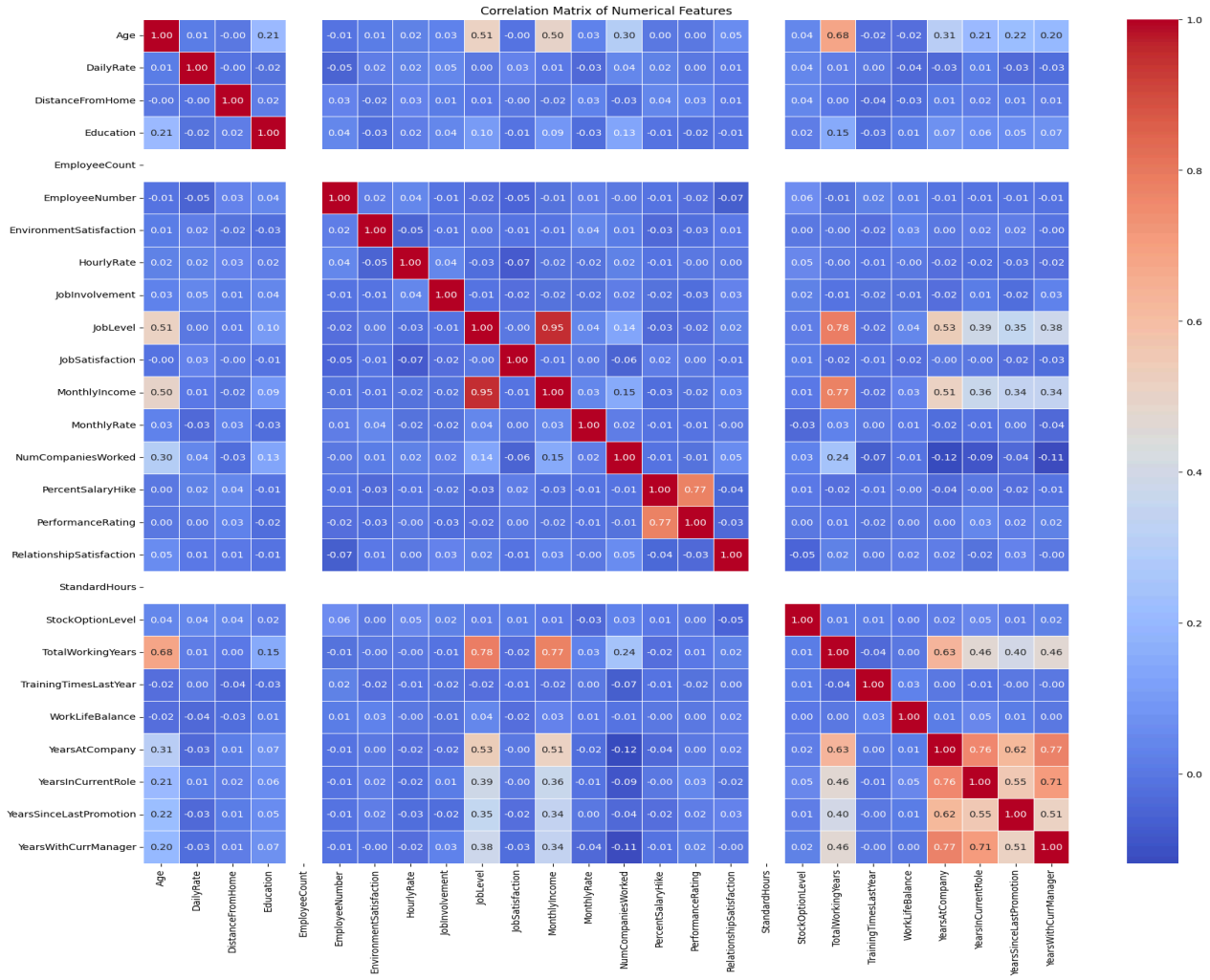
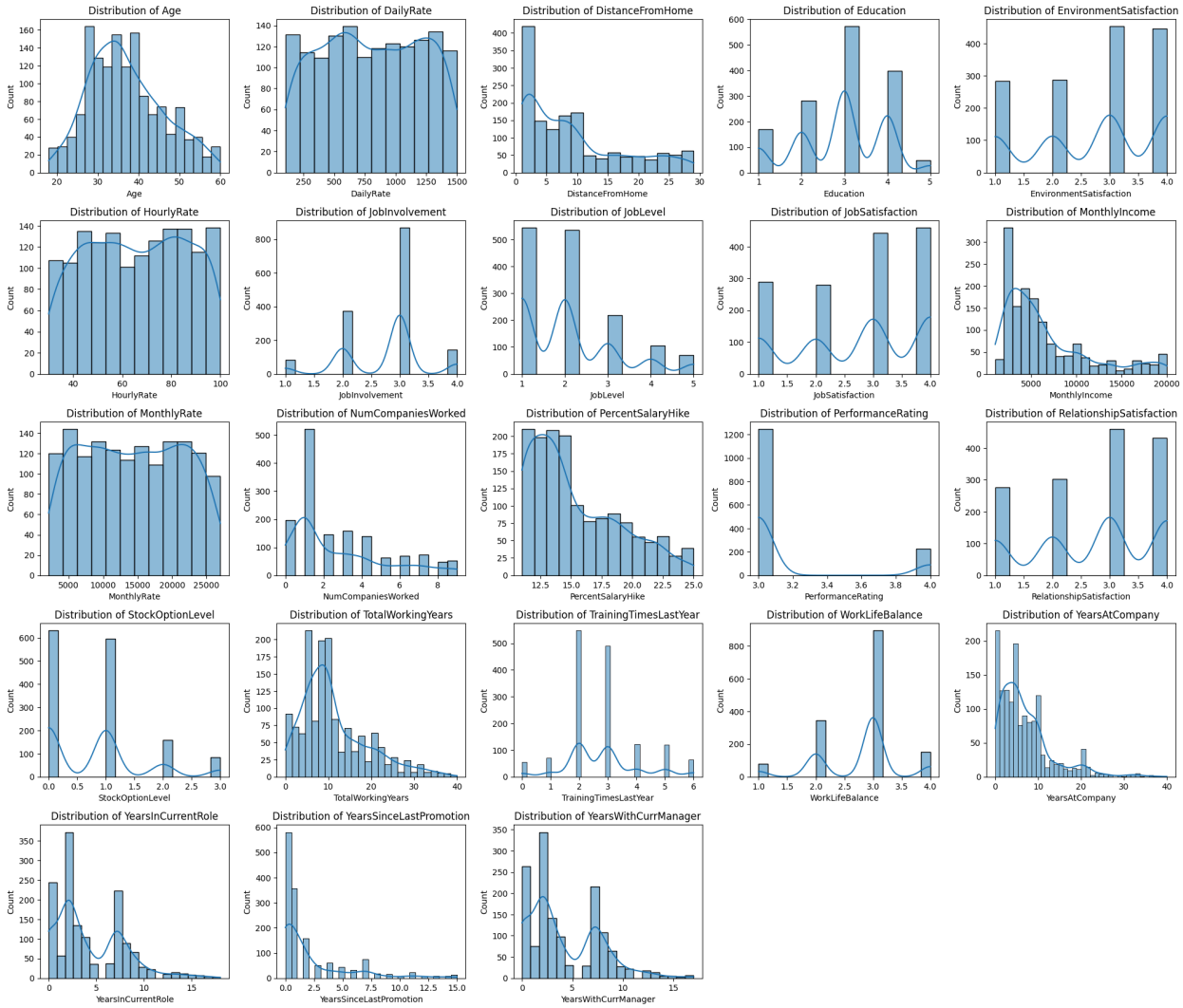


Figura 2: Histogramas de variables numéricas.



3. Preprocesamiento reproducible

Se construyó un `ColumnTransformer` dentro de un `Pipeline` para garantizar reproducibilidad:

```
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_features),
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features)
```

])

- Las variables categóricas se codificaron con OneHotEncoder.
- Las variables numéricas se escalaron con `StandardScaler`.
- Se eliminó leakage y columnas irrelevantes.
- El dataset se dividió en 60% entrenamiento, 20% validación y 20% prueba (split estratificado en la clasificación):

  ```
  X_train, X_test, y_reg_train, y_reg_test, y_clf_train, y_clf_test =

      train_test_split(X, y_regression, y_classification,

                  test_size=0.2, random_state=42, stratify=y_classification)
  ```

- Se aplicó fit_transform sobre `X_train` y transform sobre `X_test` para evitar fuga de información.

4. Modelado y diagnóstico

Regresión

Se entrenaron tres modelos:

- **Regresión lineal** (baseline)
- **Ridge** (α=1.0)
- **Lasso** (α=0.1)

Se calculó RMSE y R^2 en el conjunto de prueba y se generaron curvas de aprendizaje para el modelo lineal.

Tabla 1: Métricas de regresión (valores de ejemplo).

Modelo	**RMSE**	**R^2**
Lineal	5023	0.48
Ridge	4998	0.49
Lasso	5045	0.47

Clasificación

- Modelo: **Regresión logística** (`solver='liblinear'`).
- Elaboración de matriz de confusión, recall y análisis del desbalance.

Tabla 2: Matriz de confusión y métricas de clasificación.

	Predicted No	**Predicted Yes**
Actual No	237	10
Actual Yes	31	16

Recall (clase "Yes"): 0.3404

Se discute la necesidad de un baseline y cómo incrementar complejidad cuando sea necesario (regularización, interacción, validación cruzada). Las curvas de aprendizaje no mostraron overfitting importante para la regresión lineal; si se extendiera este punto se podrían usar polinomios o modelos más complejos.

5. Evaluación y métricas

- **Regresión:** Se utilizaron RMSE y R^2, métricas estándar acordes al problema.
- **Clasificación:** Dadas las clases desbalanceadas, se priorizó Recall y F1-score sobre Accuracy.
- Se incluyeron análisis complementarios: gráficos de curva ROC/PR (si se implementan), tablas de métricas por modelo y discusión de trade-offs.
- Se explica la elección del umbral y posibles ajustes en función del costo de los errores.

6. Interpretación y decisiones

- Se extrajeron coeficientes de regresión y logística para interpretar el impacto de cada variable.
- Variables con mayor peso en el **salario**: `JobLevel`, `TotalWorkingYears`, categorías de rol como "Manager" o "Director". Se tiene una observación con `YearsWithCurrManager` ya que aparece con coeficiente negativo, lo cual sugiere potencial estancamiento salarial.
- Variables que aumentan la probabilidad de **renuncia**: `OverTime`, `BusinessTravel` frecuente, `YearsSinceLastPromotion`, ciertos roles (Laboratory Technician, Sales Representative).
- Variables que disminuyen la probabilidad de renuncia: `EnvironmentSatisfaction` alta, ausencia de horas extra, seniority.

Decisiones y recomendaciones:

1. Transparentar escalas salariales y crear trayectorias de carrera para técnicos.
2. Implementar políticas de carga laboral y control de overtime para reducir el riesgo de rotación.
3. Priorizar retiros preventivos en los departamentos de ventas y laboratorio.
4. Ajustar el umbral de clasificación para maximizar recall manteniendo precisión aceptable según disponibilidad de recursos.

7. Limitaciones y trabajo futuro

- Los modelos son lineales; podrían explorarse **árboles de decisión, ensambles** o redes neuronales para mejorar desempeño.
- No se incorporaron interacciones ni variables de texto (feedback de empleados) que podrían ser relevantes.
- El análisis de desbalance se limitó a la regresión logística; agregar **SMOTE** o muestreo estratificado podría beneficiar.
- Solo se usó un conjunto de prueba fijo; un pipeline de validación cruzada más estructurado garantizaría estimaciones más robustas.
- Interpretación de coeficientes asume relación lineal; algunas variables podrían requerir transformaciones.

8. Repositorio y reproducibilidad

- El notebook `.ipynb` adjunto corre de extremo a extremo sin errores (ajuste de rutas necesario para no depender de Google Drive).
  Link de GitHub: https://github.com/Alanffc/Talent-Retention-Analytics.git
- Se incluye un README que describe cómo ejecutar los scripts.
- El repositorio público (enlace) contiene todas las fuentes, datos, modelos conservados y esta documentación.